

■ Détection de Casques EPI — YOLOv8

Documentation Technique — Mini-Projet 2 | IA Appliquée

■ Aperçu du Projet

Ce projet implémente un pipeline complet de détection d'Équipements de Protection Individuelle (EPI) à l'aide de **YOLOv8**, pour détecter :

■ **Helmet (Casque)**

■ **Head (Tête nue)**

Le notebook a été conçu et exécuté dans **Google Colab** et couvre l'ensemble du workflow :

- **Étape 1** — Vérification du GPU
- **Étape 2** — Installation des dépendances
- **Étape 3** — Téléchargement du dataset depuis Kaggle
- **Étape 4** — Conversion des annotations XML (VOC) → format YOLO (.txt)
- **Étape 5** — Division du dataset (train / validation / test)
- **Étape 6** — Visualisation et vérification des données
- **Étape 7** — Entraînement YOLOv8
- **Étape 8** — Évaluation du modèle
- **Étape 9** — Test sur images non vues
- **Étape 10** — Export du modèle entraîné (best.pt)

■ Dataset

Dataset utilisé : **Hard Hat Detection Dataset** de Kaggle.

Source :

<https://www.kaggle.com/datasets/andrewmvd/hard-hat-detection>

Le dataset contient des images de chantiers de construction avec des annotations Pascal VOC XML. Ce projet ne conserve que les deux classes suivantes :

Classe	ID
helmet	0
head	1

■■ Prérequis

Installation des dépendances :

```
pip install ultralytics kaggle
```

Bibliothèques principales utilisées : ultralytics, OpenCV, matplotlib, PIL, numpy, xml.etree.ElementTree.

■ Lancement du Projet

1■■ Vérifier le GPU

```
!nvidia-smi
```

2■■ Installer les dépendances

```
!pip install ultralytics kaggle --quiet
```

3■■ Configurer l'API Kaggle

Uploadez votre fichier kaggle.json dans Google Colab :

```
from google.colab import files
files.upload()
```

Placez le fichier dans :

```
/root/.kaggle/kaggle.json
```

4■■ Télécharger le Dataset

```
!kaggle datasets download -d andrewmvd/hard-hat-detection -p /content/ --unzip
```

■■ Le dataset (~1.5 Go) se décompresse directement dans /content/. Le notebook détecte automatiquement les chemins des images et annotations.

5■■ Conversion VOC XML → Format YOLO

Le notebook convertit les annotations XML en annotations YOLO .txt.

Exemple d'annotation YOLO :

```
0 0.512 0.438 0.211 0.317
```

Format : <class_id> <x_center> <y_center> <largeur> <hauteur> (coordonnées normalisées entre 0 et 1)

6 ■■ Division du Dataset

Ensemble	Pourcentage
Train	70%
Validation	20%
Test	10%

7 ■■ Configurer data.yaml

```
path: /content/dataset
```

```
train: train/images
```

```
val:   valid/images
```

```
test:  test/images
```

```
nc: 2
```

```
names:
```

```
  0: helmet
```

```
  1: head
```

8 ■■ Entraîner YOLOv8

Modèle utilisé : YOLOv8 Nano (recommandé dans le cahier des charges)

Paramètre	Valeur
Modèle	YOLOv8 Nano
Epochs	30
Image Size	640
Batch Size	16
Optimiseur	SGD
Patience	10 (early stopping)

```
results = model.train(  
    data      = YAML_PATH,  
    epochs   = 30,  
    imgsz    = 640,  
    batch    = 16,  
    name     = "helmet_detection",  
    optimizer = "SGD"  
)
```

■ Évaluation

Le notebook évalue le modèle entraîné à l'aide des métriques suivantes :

- • mAP@0.5
- • Précision
- • Rappel
- • Matrice de confusion

YOLO génère automatiquement les courbes de perte, les courbes Précision/Rappel et la matrice de confusion.

```
metrics = best_model.val()
```

■ Inférence sur Images de Test

Le notebook effectue l'inférence sur des images aléatoires du set de test et dessine les bounding boxes avec le code couleur suivant :

- • ■ Helmet — boîte VERTE
- • ■ Head — boîte ROUGE

```
results = best_model(image)
```

■ Export du Modèle Entraîné

Le modèle entraîné est exporté sous le nom best.pt. Ce fichier peut être utilisé dans :

- • Applications Streamlit
- • APIs Flask/FastAPI
- • Détection temps réel (webcam)
- • Déploiement edge AI

```
files.download(BEST_MODEL_PATH)
```

■ Résumé du Pipeline

Dataset Kaggle (5000 images + annotations XML)

↓

Conversion VOC XML → Format YOLO (.txt)

↓

Division 70% train / 20% valid / 10% test
↓
Fine-tuning YOLOv8n (pré-entraîné COCO)
↓
Évaluation mAP@0.5 + Précision + Rappel
↓
Inférence + visualisation
↓
Export best.pt → app.py (Streamlit)

■ ■ Lancement de l'Application Streamlit

Après l'entraînement et le téléchargement de best.pt, créez un dossier de projet local :

```
mkdir ppe_project  
cd ppe_project
```

Placez ces fichiers dans le dossier :

```
ppe_project/  
■ ■ ■ app.py  
■ ■ ■ best.pt  
■ ■ ■ requirements.txt
```

■ Installation des dépendances

Créez un fichier requirements.txt :

```
ultralytics>=8.2.0  
streamlit>=1.35.0  
opencv-python-headless>=4.9.0  
pandas>=2.2.0  
openpyxl>=3.1.2  
Pillow>=10.0.0  
numpy>=1.26.0  
torch>=2.2.0  
torchvision>=0.17.0
```

```
pip install -r requirements.txt
```

■ Démarrer l'application

```
streamlit run app.py
```

L'application s'ouvre automatiquement dans votre navigateur :

```
http://localhost:8501
```

■ Fonctionnalités de l'Application

- ■ Détection EPI sur image unique
- ■ Mode audit par lot (plusieurs photos)

- • ■ Détection casque / sans casque
- • ■ Calcul du taux de conformité (%)
- • ■ Export du rapport Excel
- • ■ Visualisation des bounding boxes en temps réel

■ Améliorations Possibles

- • Détection temps réel via webcam
- • Classification de la couleur du casque
- • Suivi des travailleurs (tracking)

■ Technologies Utilisées

Technologie	Rôle
Python 3.10+	Langage principal
Google Colab	Environnement d'entraînement
YOLOv8	Détection d'objets
OpenCV	Traitement d'images
Kaggle API	Téléchargement du dataset
Ultralytics	Framework YOLO
Streamlit	Interface utilisateur web